



Learning Tree-Structured Distributions

Fadwa Idlahcen^(✉) 

Faculty of Electrical Engineering, Czech Technical University in Prague,
Karlovo namesti 13, Prague 12000, Czech Republic
idlahfad@fel.cvut.cz

Abstract. Tree-structured probabilistic models offer a balance between interpretability and tractability, making them a cornerstone in density estimation and graphical model learning. In this work, we present an empirical evaluation of three recent approaches for learning tree distributions, benchmarking them against the classic Chow-Liu algorithm. Although Chow-Liu is decades old, our results show that it continues to outperform or match the newer methods in most scenarios, including both synthetic and real-world datasets.

A key contribution of this study is the implementation and practical evaluation of two online learning algorithms, which had not previously been tested in practice or compared directly with the other methods we use. We analyze performance across multiple dimensions: realizability of the data distribution, alphabet size, and dataset scale. Our findings reaffirm the competitiveness of the Chow-Liu algorithm while also offering a comparison among the three methods, shedding light on their respective strengths and limitations across different regimes.

Keywords: Tree-structured distribution · Chow-Liu algorithm · Online learning

1 Introduction

Bayesian networks (BNs) are widely used across various domains due to their ability to model causal relationships among random variables—for instance, in medicine [9] and gene regulatory networks [10]. However, learning the structure of BNs from data is an NP-hard problem [6]. Additionally, performing inference with BNs is computationally non-trivial [8].

To address the challenges of learning and inference in BNs, numerous approaches have been proposed. While many of these methods yield strong results, they often involve a trade-off between computational tractability and model expressiveness. Certain BN structures, such as Naive Bayes and tree-structured networks, offer computational efficiency but at the cost of reduced expressiveness. Nonetheless, these simpler distributions are frequently employed in machine learning—for example, in variational inference—due to their practicality. Tree-structured models are particularly common, as they enable efficient probabilistic inference [15]. Furthermore, more expressive models can be

constructed by building on tree structures, such as through mixtures-of-trees approaches [14].

In this paper, we compare several methods for learning tree-structured distributions. A foundational approach in this domain is the Chow-Liu algorithm [7], introduced in 1968, which efficiently identifies the tree structure and parameters that maximize the likelihood of the observed data. Our primary objective is to evaluate the efficiency of recent methods relative to the Chow-Liu baseline. These methods are theoretically state-of-the-art, offering theoretical guarantees, and we aim to assess their empirical performance through experimentation:

- Distribution Learning Meets Graph Structure Sampling [2]
- Online Forest Density Estimation (OFDE) [11]
- Mixture of All Trees (MoAT)¹ [16]

The main contribution of this work is a practical comparison of these algorithms² We aim to highlight their empirical strengths and limitations, particularly in light of the lack of experimental results in [2] and the unavailability of implementation code for [11].

1.1 Tree-Structured Distributions

In this section, we introduce the notation used throughout the paper and outline the learning framework for tree-structured distributions. A Bayesian network over n discrete variables, each taking values from an alphabet size k , is represented as a directed acyclic graph (DAG) that encodes causal relationships among the variables. These dependencies are quantified through conditional probability distributions for each variable, conditioned on its parent(s) in the network [15].

Tree-structured distributions—referred to here interchangeably as trees or tree distributions—are a subclass of BNs in which the underlying DAG has a maximum in-degree of 1. Alternatively, such distributions can be represented by an undirected graph $G = (V, E)$, also known as a Markov Random Field (MRF), where V and E denote the sets of vertices and edges, respectively. The normalized joint distribution defined over this graph, as described in [14], is given by:

$$Pr_G(\mathbf{x}) = \frac{\prod_{(u,v) \in E} P_{uv}(x_u, x_v)}{\prod_{u \in V} P_u(x_u)^{deg(u)-1}} \quad (1)$$

Here, $\mathbf{x}$ denotes an assignment of values to all variables; $deg(u)$ is the degree of vertex u ; $P_{uv}(x_u, x_v)$ is the joint distribution over variables u and v ; and $P_u(x_u)$ is the marginal distribution of x_u , obtained from $P_{uv}(x_u, x_v)$ or $P_{vu}(x_v, x_u)$ by marginalizing over x_v for any $v \neq u$.

¹ While this method does not provide theoretical guarantees, we include it due to its strong empirical performance and conceptual similarity to [2].

² Our Python implementation is available at <https://github.com/Fadwaid2/Tree-distributions..>

The task of learning tree-structured distributions involves estimating both the structure and the parameters of the model that maximize the log-likelihood of a given dataset. Specifically, given a set of observations $\{x^{(1)}, x^{(2)}, \dots, x^{(T)}\}$, where T denotes the number of samples, the goal is to identify the tree model that best fits the data. In this setting, the structure is constrained to be a spanning tree over the n random variables (nodes). A spanning tree is an acyclic, connected subgraph that includes all n nodes and exactly $n - 1$ edges. The collection of all possible spanning trees over n nodes forms the spanning tree polytope, a convex hull of incidence vectors of all spanning trees. The number of such trees is exponential in n , and is given by Cayley's formula, which states that there are exactly n^{n-2} spanning trees for a complete graph with n nodes. Learning a tree-structured distribution can also be viewed as minimizing the Kullback–Leibler (KL) divergence between the empirical distribution $\hat{P}$ of the data and a candidate tree-structured distribution Q from the model class. That is, the objective becomes:

$$\min_{Q \in \mathcal{T}} KL(\hat{P}||Q) \quad (2)$$

where $\mathcal{T}$ is the set of all tree-structured distributions. In the realizable case, when the true distribution lies within $\mathcal{T}$, this corresponds to exact recovery of structure; in the agnostic case, the goal is to find the closest tree-structured approximation to a potentially non-tree distribution.

The large combinatorial space of possible spanning trees underscores the computational efficiency of algorithms like Chow-Liu, which cleverly use maximum-weight spanning tree algorithms to optimize over this exponentially large space efficiently, by reducing the KL divergence to a sum over mutual information terms.

1.2 Online Learning

Both [2] and [11] adopt an Online Learning (OL) framework. In this setting, the model learns sequentially from data by sampling from a class of candidate structures [2]. We assume that the observations $\{x^{(1)}, x^{(2)}, \dots, x^{(T)}\}$ are independent and identically distributed (i.i.d.) samples drawn from an unknown distribution P^* . At each time step t , the learner observes a new data point $x^{(t)}$ and produces an estimate $\hat{\theta}_t$, based on the previous observations $x_{1:(t-1)} := \{x^{(1)}, \dots, x^{(t-1)}\}$. The performance at time t is measured by a loss function $l(x^{(t)}, \hat{\theta}_t)$, which we denote simply as $l(\hat{\theta}_t)$ for brevity. The overall quality of the sequential estimation is evaluated using regret, which measures the cumulative discrepancy between the learner's predictions and the best possible fixed predictor in hindsight. Formally, the cumulative regret of the sequence $\{\hat{\theta}_t\}_{t=1}^T$ is defined as follows:

$$\sum_{t=1}^T l_t(\hat{\theta}_t) - \inf_{\theta \in \Theta} \sum_{t=1}^T l_t(\theta) \quad (3)$$

The objective, therefore, is to minimize the cumulative regret, with the loss function defined as the negative log-likelihood. In this context, the minimax regret

quantifies the best possible worst-case performance of a forecasting algorithm across all possible classes of N experts and all outcome sequences of length T [4]. Shtarkov [17] introduced a forecasting method based on the Normalized Maximum Likelihood (NML) distribution, which achieves the minimax regret when the time horizon T is known in advance. However, computing the NML distribution is intractable in practice, as it requires summing over an exponential number of terms [4]. As a result, mixture-of-experts approaches are often preferred due to their computational efficiency and the fact that they do not require knowledge of the time horizon T . Moreover, their theoretical regret bounds are comparable to those of the NML forecaster [4], which further motivates their widespread use.

Another widely studied forecasting strategy is Follow-the-Leader (FTL) and its randomized variant, Follow-the-Perturbed-Leader (FPL). These algorithms achieve regret bounds that grow logarithmically with the number of iterations [12].

Outside the online learning framework, the MoAT method provides an exact computation of the normalized likelihood by averaging over all n^{n-2} possible tree structures. This approach enables the use of a full mixture-of-trees model with tractable inference. Further details on each of the selected methods are provided in the following section. One approach to comparing these methods is through their sample complexity—the amount of data required to guarantee a specified level of performance. Below, we summarize the theoretical bounds as reported in the respective works.

For OFDE, the algorithm achieves minimax regret bounds of $O(nk^2 \ln T + n^2 \ln T \sqrt{T})$ in the horizon-dependent case, and $O(nk^2 \ln T + n^2 (\ln T)^2 \sqrt{T})$ in the horizon-independent case [11]. The method in [2] is analyzed under the Probably Approximately Correct (PAC) learning framework. It provides a sample complexity of $\tilde{O}(nk^2 \epsilon^{-1})$ (ignoring logarithmic factors), ensuring that the learned mixture-of-trees distribution $\hat{P}$ satisfies:

$$KL(P^* || \hat{P}) \leq 3 \min_{P \in \mathcal{T}} (KL(P^* || P)) + \epsilon \quad (4)$$

with probability at least $2/3$, where $\mathcal{T}$ denotes the class of all tree-structured distributions. For the Chow-Liu algorithm, the realizable-case PAC sample complexity has been shown in [3] to be $\tilde{O}(nk^3 \epsilon^{-1})$, indicating a factor of k worse dependence compared to the bound in [2]. As far as we are aware, no formal sample complexity guarantees have been established for MoAT. Nonetheless, MoAT remains of practical interest due to its empirical performance.

In this paper, we empirically examine how these theoretical guarantees correspond to observed performance, particularly in data-limited settings.

2 Methods

In this section, we describe the four algorithms employed in this work and provide a summary of their underlying techniques.

2.1 Chow-Liu

The Chow-Liu algorithm [7] is a classical and widely-used method for learning joint probability distributions with tree-structured dependencies. It leverages the key insight that the tree structure maximizing the likelihood corresponds to the maximum-weight spanning tree of the complete graph. In this graph, the weight of each edge represents the mutual information between the two connected random variables.

2.2 Distribution Learning Meets Graph Structure Sampling

In [2], a sampling algorithm embedded within two online learning algorithms, where the set of experts corresponds to a mixture-of-trees, is proposed. The sampling of spanning trees leverages the matrix tree theorem.

Since we employ only one of these OL algorithms in this work—the Randomized Weighted Majority (RWM)—we will refer to the method from [2] simply as RWM hereafter. This approach requires discretization to finitely represent the class of distributions. Although [4] generalizes regret bounds to infinitely large expert classes—making discretization theoretically unnecessary—this step is used in [2] to directly apply regret bounds from that work.

2.3 Online Forest Density Estimation (OFDE)

The OFDE algorithm [11] addresses the problem of learning tree-structured distributions in the OL setting. At each time step, the algorithm maintains and updates a probabilistic estimate over the space of spanning trees, and predicts a tree structure and its associated parameters to model the data.

The structure update step is performed using Follow-the-Perturbed-Leader algorithm. This method perturbs the cumulative loss associated with each potential edge using random noise and then selects a spanning tree that minimizes the perturbed loss. Importantly, this optimization is carried out over the spanning tree polytope using a greedy matroid algorithm—which in the case of trees is equivalent to Kruskal’s algorithm.

However, the edge weights in the FPL algorithm are fractional, and generating a valid tree structure requires converting these fractional solutions into integral spanning trees. To accomplish this, OFDE employs swap rounding [5], a randomized rounding procedure that samples a spanning tree from a convex combination while preserving the marginal probabilities of edge inclusion.

Following structure selection, the parameter update step estimates univariate and pairwise marginal distributions using an approach similar to Bayesian mixtures with Dirichlet priors. This leverages the additive nature of the log-likelihood function, allowing the algorithm to incrementally update sufficient statistics and compute a valid likelihood estimate for each tree.

By combining efficient combinatorial optimization with online probabilistic estimation, OFDE provides a scalable approach to learning tree-structured models.

2.4 Mixture of All Trees (MoAT)

The MoAT algorithm [16] introduces a generative model that considers all n^{n-2} possible spanning trees over n variables, operating outside the OL framework. By using a mixture-of-trees representation, the structure is fixed during training, while parameters are learned efficiently.

This tractability is achieved by assigning each tree a weight proportional to its probability within the spanning tree distribution (see Eq. 2 in [16]). Importantly, parameters are shared across all trees in the mixture since the univariate and bivariate probabilities remain consistent for each random variable.

Similar to [2], the matrix tree theorem is employed, but here it is combined with the concept of probability generating polynomials (Eq. 3 in [16]). This yields a closed-form, tractable expression for the likelihood, where the primary computational bottleneck is the calculation of the determinant of the Laplacian matrix—also encountered in the RWM algorithm [2].

MoAT is trained by maximizing the likelihood through stochastic gradient descent, with deterministic parameter initialization. For the latter, mutual information between variables is incorporated following Chow-Liu’s method, which improves the model’s performance.

3 Results and Discussion

In this study, we aim to rigorously evaluate the performance of recent structure learning methods in comparison to the classical Chow-Liu algorithm. Our goal is to understand how these modern approaches behave under varying conditions, both synthetic and real. To this end, we formulate four key research questions that guide our experimental design and analysis:

- Q1. Comparative Performance: Do the three recent learning methods outperform the classical Chow-Liu algorithm?
- Q2. Robustness Across Data Generating Processes: How does the performance of OL methods differ between the realizable case, where data are generated from a true tree-structured distribution, and the agnostic case, where data arise from more complex non-tree graphs?
- Q3. Scalability with Alphabet Size: Which of the OL methods demonstrates greater robustness and performance as the alphabet size (i.e., the number of discrete variable states) increases?
- Q4. Behavior on Large Datasets: As the number of samples grows, which method scales more effectively in terms of accuracy and computational efficiency?

To address these questions, we design a comprehensive collection of experiments that includes both controlled synthetic datasets—allowing us to systematically vary underlying structures, alphabet sizes, and sample sizes—and real-world datasets, which provide a test of generalization and practical utility.

3.1 Synthetic Datasets

To systematically evaluate the performance of the learning methods, we construct synthetic datasets using discrete random variables across two primary regimes: low-dimensional ($T \gg n$) and high-dimensional. The specific combinations of the number of variables n and the number of samples T used in our experiments are:

- $(n, T) = (5, 20), (5, 50), (10, 20), (10, 50), (20, 50), (20, 100), (50, 500), (50, 1000), (100, 500), (100, 1000)$

We generate datasets under two generative models:

- Tree-structured distributions
- General Markov Random Fields (MRFs) based on the Erdős-Rényi random graph model.

To introduce additional complexity and simulate real-world imperfections, we inject 50% noise by randomly shuffling the observed values of the variables.

We consider discrete alphabets with sizes $k \in \{2, 5, 10\}$. However, since the MoAT algorithm supports only binary variables, it is excluded from experiments involving $k \in \{5, 10\}$. Additionally, to ensure a fair comparison with other methods that recover single tree structures, we adapt the use of MoAT: rather than leveraging the full mixture of trees, we retrieve (using Kruskal’s algorithm) and evaluate a single tree structure from the learned mixture.

Each configuration is run 5 times to account for variability in algorithms with stochastic components (namely, OFDE and RWM). We report the mean and standard deviation of the results across these runs.

For the binary case ($k = 2$), we present results for both tree-structured and MRF-generated data using two metrics:

- Structural Hamming Distance (SHD): measures structural accuracy; lower values indicate better alignment with the ground-truth graph.
- Log-likelihood (LL): evaluated on held-out test data sampled from the true generative model.

These results are summarized in Tables 1 and 2, respectively.

For the non-binary cases ($k \in \{5, 10\}$), we report SHD and log-likelihoods in Tables 3 and 4. Across all tables, bolded values indicate the best-performing method for a given metric and setting.

3.2 Real-World Datasets

In addition to synthetic data, we evaluate the performance of the structure learning methods on real-world datasets drawn from the Twenty Datasets benchmark suite for density estimation [18]. These datasets cover a range of domains and data characteristics, providing a robust testbed for empirical validation. A summary of the dataset properties (number of variables and samples) is provided in Table 5.

Table 1. SHD and average LL for $k = 2$ - data from tree.

	Chow-Liu		RWM		OFDE		MoAT	
	SHD	LL	SHD	LL	SHD	LL	SHD	LL
(5,20)	4	-2.76	7.2 ± 1.10	-2.92 ± 0.11	4.8 ± 1.10	-2.85 ± 0.11	2	-2.74
(5,50)	2	-2.86	5.2 ± 3.03	-2.92 ± 0.05	4.8 ± 2.68	-2.92 ± 0.07	2	-2.85
(10,20)	12	-6.61	13.6 ± 2.97	-7.22 ± 0.16	15.6 ± 1.67	-6.88 ± 0.33	10	-6.74
(10,50)	14	-6.24	13.2 ± 1.79	-6.44 ± 0.19	14.4 ± 0.89	-6.44 ± 0.06	14	-6.35
(20,50)	32	-13.27	32.8 ± 3.35	-13.78 ± 0.21	35.6 ± 0.89	-13.59 ± 0.17	38	-13.48
(20,100)	36	-13.16	32.8 ± 1.79	-13.37 ± 0.07	33.2 ± 3.35	-13.41 ± 0.23	36	-13.17
(50,500)	94	-34.02	93.6 ± 1.67	-34.12 ± 0.05	94.0 ± 2.45	-34.08 ± 0.00	94	-34.06
(50,1000)	98	-34.00	94.4 ± 2.61	-34.05 ± 0.02	95.2 ± 3.03	-34.02 ± 0.02	90	-34.01
(100,500)	194	-68.68	194.0 ± 1.41	-68.99 ± 0.07	194.4 ± 2.61	-68.81 ± 0.06	184	-68.83
(100,1000)	194	-68.67	196.0 ± 2.00	-68.76 ± 0.02	195.2 ± 1.10	-68.72 ± 0.01	192	-68.68

Table 2. SHD and average LL for $k = 2$ - data from MRF.

	Chow-Liu		RWM		OFDE		MoAT	
	SHD	LL	SHD	LL	SHD	LL	SHD	LL
(5,20)	5	-3.17	5.0 ± 0.00	-5.25 ± 2.97	5.8 ± 1.10	-7.16 ± 2.33	3	-3.07
(5,50)	5	-2.90	5.0 ± 0.00	-3.03 ± 0.11	4.2 ± 1.10	-2.86 ± 0.05	7	-2.97
(10,20)	28	-6.81	26.0 ± 2.00	-6.72 ± 0.19	25.6 ± 3.58	-6.77 ± 0.35	26	-6.59
(10,50)	29	-6.56	24.6 ± 1.67	-6.60 ± 0.04	27.4 ± 2.97	-6.58 ± 0.19	23	-6.65
(20,50)	101	-13.33	101.0 ± 4.00	-13.78 ± 0.12	101.4 ± 3.58	-13.40 ± 0.18	105	-13.35
(20,100)	114	-13.31	108.4 ± 2.97	-13.50 ± 0.05	110.0 ± 5.83	-13.40 ± 0.08	110	-13.41
(50,500)	755	-34.00	751.8 ± 7.16	-34.06 ± 0.02	745.4 ± 9.74	-34.02 ± 0.03	749	-34.02
(50,1000)	746	-33.98	727.2 ± 3.90	-34.04 ± 0.01	734.0 ± 9.38	-34.02 ± 0.03	728	-34.00
(100,500)	2996	-68.69	2996.0 ± 8.60	-68.82 ± 0.03	2994.4 ± 11.61	-68.83 ± 0.02	2994	-68.71
(100,1000)	2930	-68.71	2935.2 ± 9.44	-68.73 ± 0.01	2933.6 ± 5.73	-68.76 ± 0.02	2938	-68.68

Table 3. SHD and average LL for alphabet size $k = 5$ and $k = 10$ - data from tree structure.

		Chow-Liu		RWM		OFDE	
		SHD	LL	SHD	LL	SHD	LL
(5,20)	k=5	4	-30.22	4.4 ± 2.61	-41.94 ± 6.40	6.0 ± 2.00	-34.09 ± 1.89
	k=10	6	-49.45	4.4 ± 2.19	-66.25 ± 10.54	3.6 ± 1.67	-71.54 ± 4.63
(5,50)	k=5	4	-11.56	4.4 ± 1.67	-14.03 ± 1.13	3.6 ± 1.67	-13.56 ± 1.38
	k=10	6	-32.11	5.6 ± 1.67	-49.18 ± 7.33	5.2 ± 1.10	-45.09 ± 4.92
(10,20)	k=5	12	-57.99	15.2 ± 1.10	-83.80 ± 12.63	14.0 ± 1.41	-83.29 ± 11.95
	k=10	16	-105.09	14.8 ± 2.28	-130.10 ± 9.83	15.2 ± 1.79	-140.47 ± 6.95
(10,50)	k=5	16	-22.99	16.4 ± 1.67	-34.85 ± 5.12	14.0 ± 2.00	-32.74 ± 2.34
	k=10	16	-54.79	13.6 ± 2.61	-99.77 ± 5.83	16.4 ± 2.19	-100.95 ± 12.17
(20,50)	k=5	36	-92.15	33.2 ± 4.15	-93.47 ± 7.04	33.2 ± 2.68	-88.79 ± 2.70
	k=10	32	-137.09	36.8 ± 1.79	-238.88 ± 12.75	34.8 ± 2.68	-228.73 ± 9.94
(20,100)	k=5	34	-32.42	33.6 ± 1.67	-42.97 ± 3.22	32.8 ± 3.63	-43.07 ± 3.10
	k=10	30	-74.47	35.6 ± 1.67	-156.76 ± 9.21	33.6 ± 2.19	-159.94 ± 4.53
(50,500)	k=5	98	-78.73	96.0 ± 2.45	-79.11 ± 0.07	93.6 ± 3.85	-79.09 ± 0.16
	k=10	90	-113.95	96.4 ± 1.67	-131.30 ± 2.03	89.6 ± 3.29	-131.39 ± 1.04
(50,1000)	k=5	94	-78.48	94.4 ± 1.67	-78.64 ± 0.03	94.4 ± 1.67	-78.74 ± 0.07
	k=10	96	-109.38	94.0 ± 2.83	-111.25 ± 0.58	94.4 ± 3.85	-111.79 ± 0.35
(100,500)	k=5	194	-159.33	192.0 ± 2.45	-160.19 ± 0.14	193.6 ± 2.61	-160.38 ± 0.16
	k=10	194	-227.98	195.2 ± 1.79	-262.33 ± 4.49	192.8 ± 3.03	-263.89 ± 5.14
(100,1000)	k=5	196	-158.53	193.2 ± 1.10	-158.97 ± 0.06	193.2 ± 2.28	-159.12 ± 0.06
	k=10	194	-221.77	194.4 ± 3.58	-223.68 ± 0.56	194.0 ± 2.45	-226.48 ± 0.48

Table 4. SHD and average LL for alphabet size $k = 5$ and $k = 10$ - data from MRF.

		Chow-Liu		RWM		OFDE	
		SHD	LL	SHD	LL	SHD	LL
(5,20)	k=5	8	-29.59	5.6 ± 0.89	-26.28 $\pm$ 1.34	3.6 $\pm$ 0.89	-27.78 ± 1.50
	k=10	5	-40.1	3.8 $\pm$ 1.10	-46.44 ± 5.17	5.4 ± 1.67	-50.40 ± 4.60
(5,50)	k=5	7	-12.49	6.2 ± 1.10	-15.27 ± 1.77	4.2 $\pm$ 1.10	-14.55 ± 1.29
	k=10	5	-17.83	6.6 ± 0.89	-25.40 ± 3.51	4.6 $\pm$ 1.67	-29.37 ± 4.91
(10,20)	k=5	25	-89.73	26.6 ± 2.19	-83.16 $\pm$ 8.02	24.6 $\pm$ 2.61	-85.45 ± 6.35
	k=10	24	-52.45	26.4 ± 1.67	-77.62 ± 13.83	26.4 ± 1.67	-111.77 ± 8.88
(10,50)	k=5	25	-21.5	22.6 ± 0.89	-32.31 ± 2.58	21.8 $\pm$ 1.10	-36.07 ± 2.27
	k=10	22	-58.82	23.2 ± 2.28	-69.35 ± 6.69	24.8 ± 1.79	-85.51 ± 6.76
(20,50)	k=5	112	-68.88	110.8 ± 2.28	-84.72 ± 4.17	110.4 $\pm$ 3.58	-93.80 ± 9.16
	k=10	108	-135.44	109.6 ± 6.23	-210.82 ± 9.53	109.2 ± 5.76	-253.01 ± 11.51
(20,100)	k=5	120	-33.46	120.0 ± 3.16	-40.95 ± 2.09	116.4 $\pm$ 7.13	-38.65 ± 2.57
	k=10	110	-75.38	111.2 ± 1.79	-120.70 ± 8.03	112.4 ± 3.29	-147.14 ± 12.75
(50,500)	k=5	715	-78.5	699.4 ± 3.29	-77.29 $\pm$ 0.19	696.2 $\pm$ 7.69	-78.11 ± 0.29
	k=10	740	-114.81	737.2 $\pm$ 6.72	-124.54 ± 1.13	742.0 ± 7.75	-132.01 ± 1.79
(50,1000)	k=5	723	-77.6	703.4 ± 4.34	-76.49 $\pm$ 0.11	699.8 $\pm$ 4.82	-77.42 ± 0.14
	k=10	750	-110.12	740.4 $\pm$ 3.58	-109.05 $\pm$ 0.40	742.8 ± 3.63	-111.94 ± 0.39
(100,500)	k=5	2966	-158.83	2948.4 $\pm$ 14.79	-157.88 $\pm$ 0.11	2949.2 ± 8.44	-159.20 ± 0.26
	k=10	2858	-230.23	2858.0 ± 10.10	-250.15 ± 2.05	2862.0 ± 9.70	-258.87 ± 2.65
(100,1000)	k=5	3001	-158.78	2991.4 ± 4.77	-157.22 $\pm$ 0.07	2991.0 $\pm$ 5.10	-158.26 ± 0.17
	k=10	2874	-226.09	2863.2 $\pm$ 6.10	-224.05 $\pm$ 0.53	2866.0 ± 6.00	-227.63 ± 0.54

We apply all four methods under study to each dataset. To account for variability due to algorithmic randomness (particularly for stochastic methods such as OFDE and RWM), we repeat each experiment five times, and report the average log-likelihood on held-out test data.

The results, presented in Table 6, reflect the quality of the learned models in terms of generalization performance. Higher log-likelihood values indicate better fit to the underlying data distribution.

Table 5. Real-world datasets.

Dataset Name	Train set	Valid set	Test set	# vars	Density
accidents [18]	12758	1700	2551	111	0.291
baudio [13]	15000	2000	3000	100	0.199
bnetflix [13]	15000	2000	3000	100	0.541
dna [18]	1600	400	1186	180	0.253
jester [13]	9000	1000	4116	100	0.608
kdd [13]	180092	19907	34955	65	0.008
nlts [13]	16181	2157	3236	16	0.332
plants [13]	17412	2321	3482	69	0.180
pumsb star [18]	12262	1635	2452	163	0.270
tretail [18]	22041	2938	4408	135	0.024

Table 6. Average LL of real data. ML: memory limit; TL: time limit.

	Chow-Liu	MOAT	OFDE	RWM
accidents	-35.72	-39.43	-38.32 ± 1.53	-44.57 ± 0.74
baudio	-46.18	-50.28	-47.38 ± 0.74	-48.46 ± 0.44
bnetflix	-64.13	-62.88	-63.08 ± 0.42	-63.56 ± 0.21
dna	-99.22	-95.85	-100.02 ± 0.16	-88.93 $\pm$ 0.53
jester	-63.48	-64.18	-61.01 $\pm$ 0.34	-61.96 ± 0.03
kdd	-1.89	-1.38	-2.43 ± 0.18	ML
nlts	-7.63	-7.76	-7.20 $\pm$ 0.25	-8.01 ± 0.00
plants	-30.85	-32.02	-28.46 $\pm$ 1.11	-31.38 ± 0.11
pumsb star	-43.21	-48.99	-52.16 ± 2.42	TL
tretrain	-7.12	-6.83	-7.28 ± 0.51	ML

3.3 Hyperparameters

For both synthetic and real-world datasets, we adopt established hyperparameter settings from prior work, if existing, to ensure a fair, reproducible, and unbiased evaluation. No additional tuning was performed on a per-dataset basis to avoid overfitting and to maintain comparability with previously reported results.

- **MoAT**: We follow the recommended settings from [16]. For datasets with fewer than 500 variables, we use a batch size of 1024 and a learning rate of 0.05. For larger datasets (≥ 500 variables), we use a batch size of 64 and a learning rate of 0.01. All models are trained for up to 50 iterations with early stopping enabled. To enable comparison with other single-tree methods, we use Kruskal’s algorithm on the final edge weight matrix of the learned mixture to obtain a single tree structure.
- **RWM**: We use the theoretical parameters from [2]. Specifically, we set $\epsilon = 0.9$, which controls the approximation guarantee, and compute parameters: $\eta = \sqrt{\frac{8 \log N}{T}}$, $\tau = \frac{\epsilon/2}{4k^2}$, and $\gamma = \frac{\epsilon/2}{4}$.
- **OFDE**: We use the default hyperparameters from the horizon-independent setting in [11], with $\alpha_t = \frac{1}{4\sqrt{2t}}$ and $\beta_t = \frac{1}{n} \sqrt{2/t}$, where t is the time step.

3.4 Discussion and Conclusions

To compare the performance of the evaluated methods across synthetic and real-world datasets, we adopt Bayesian signed-rank tests as described in [1]. Unlike traditional null hypothesis testing, Bayesian analysis quantifies the posterior probability that one method outperforms another, providing richer insights into relative algorithmic performance.

We compute the posterior distribution of the mean difference in average log-likelihoods between methods and visualize the results as histograms. Each

histogram represents the distribution of sampled differences between two methods, where each bar corresponds to the relative frequency (or probability mass) of a specific range of performance differences. The height of a bar reflects how likely it is that the true difference in performance falls within that interval, based on the posterior distribution. A Region of Practical Equivalence (ROPE), set to 0.5, marks the threshold below which performance differences are considered negligible. Histograms are shown in Figs. 1–5.

Our observations support the following conclusions:

Q1: Do the three methods outperform Chow-Liu, given that they are more recent?

Overall, the Chow-Liu algorithm outperforms the more recent methods on the synthetic datasets. It consistently achieves the highest log-likelihood and the lowest structural error (SHD), especially when the underlying data is generated from tree-structured models (Fig. 1, 3(a,b,c), 4(a,b,c)).

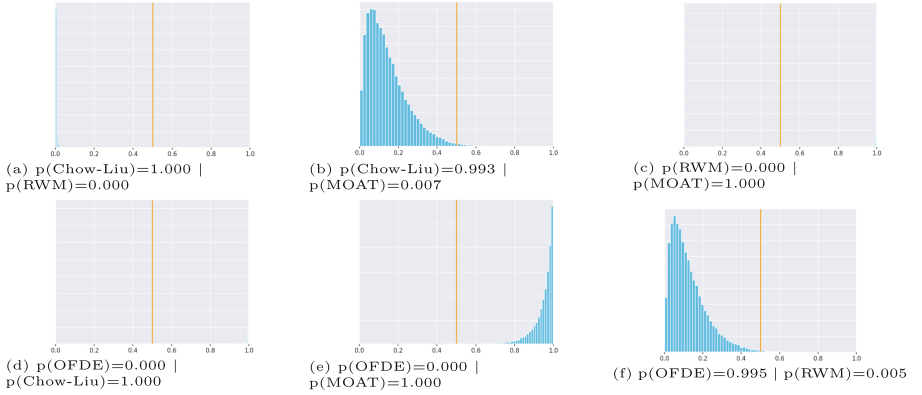


Fig. 1. Histograms for data from tree - $k=2$

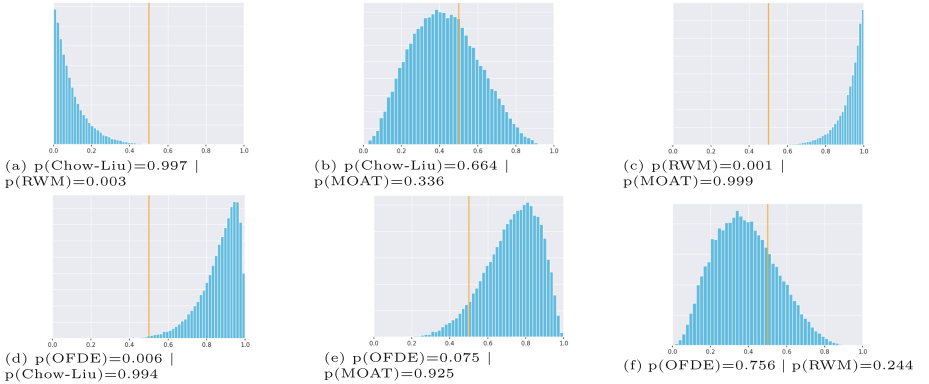


Fig. 2. Histograms for data from MRF - $k=2$

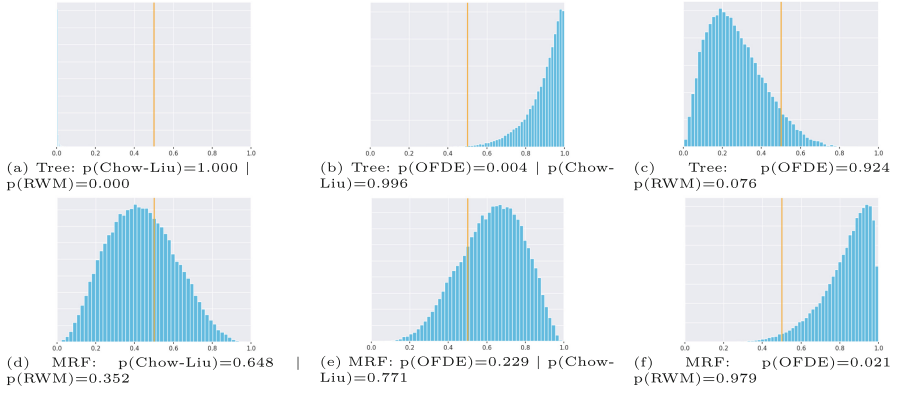


Fig. 3. Histograms for $k = 5$. Top row: data from tree; bottom row: data from MRF.

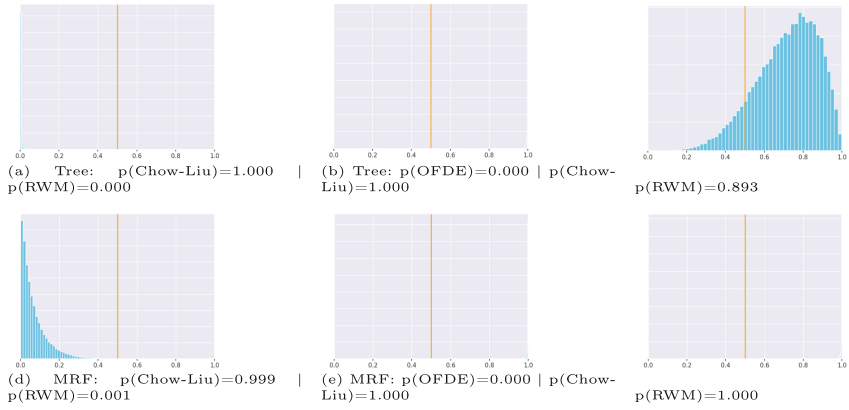


Fig. 4. Histograms for $k = 10$. Top row: data from tree; bottom row: data from MRF.

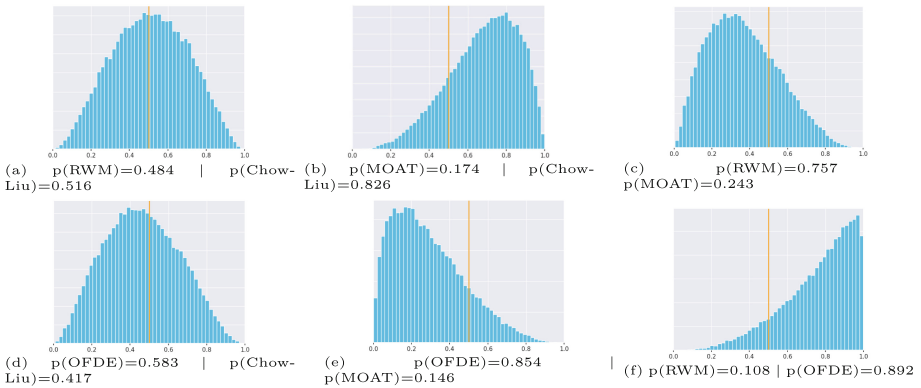


Fig. 5. Histograms for real data

While Chow-Liu’s performance degrades slightly on real-world datasets (Fig. 5), it still remains highly competitive. Notably, in some real data settings, OFDE slightly outperforms Chow-Liu, indicating strong generalization performance for OFDE in practical scenarios. This result suggests that Chow-Liu remains a robust baseline, but OFDE may be preferable when working with noisy real-world data.

Q2: What are the differences in performance between the realizable and agnostic cases in OL?

In synthetic experiments, we distinguish between the realizable case and the agnostic case. Here, the performance gap between OFDE and RWM becomes more evident.

- In realizable settings, OFDE consistently outperforms RWM (Figs. 1, 3(a,b,c)), except when the alphabet size is larger (Fig. 4(a,b,c)).
- In agnostic settings, where the true distribution may lie outside the hypothesis class, RWM performs better than OFDE (Figs. 3(d,e,f), 4(d,e,f)), except for small alphabet sizes (Fig. 2).

These findings confirm that OFDE is more reliable when model assumptions match the data, while RWM may be better suited for more flexible settings, addressing Q2.

Q3: Which of the OL methods performs better as the alphabet size grows?

Our results indicate that RWM scales better with increasing alphabet size. At $k = 5$ and especially $k = 10$, RWM surpasses OFDE in most settings. This trend is reversed at $k = 2$, where OFDE maintains a consistent advantage.

This suggests that RWM’s design, which incorporates exploration and probabilistic updates, adapts more effectively to larger state spaces.

Both Q2 and Q3 justify why OFDE performed better than RWM on real datasets in Table 6, given that the alphabet size is small ($k = 2$) and the structures are more complex—suggested by high densities (Table 5). An interesting observation is that RWM outperformed the other methods on one real-world dataset (“dna”), which stands out as the only high-dimensional dataset in the benchmark, characterized by $n/T \geq 0.1$. A future avenue could be to test on more high dimensional real datasets to check this hypothesis.

Q4: Which method is better when datasets are large?

In very high-dimensional or large-scale settings, the online learning methods (OFDE and RWM) struggle, particularly in terms of scalability and stability. Consequently, we exclude large datasets (with more than 200 variables) from the Twenty Datasets benchmark due to computational costs in memory and time.

In contrast, Chow-Liu and MoAT handle large datasets more effectively, especially in terms of runtime and convergence. Chow-Liu, being a greedy algorithm with a closed-form mutual information computation, scales linearly with the number of variables. MoAT, although slower, remains tractable with appropriate batching.

Therefore, for large-scale applications, Chow-Liu or MoAT are significantly more advantageous, effectively addressing Q4. To pursue the proposed future

direction—namely, testing on more high-dimensional real-world datasets—it would be necessary to adapt both OFDE and RWM to better handle such settings.

In conclusion:

- Chow-Liu remains the strongest performer across most settings, especially for tree-structured and moderate-sized data.
- OFDE performs best in realizable and low-alphabet scenarios.
- RWM shines in agnostic settings and scales better with alphabet size.
- Although MoAT is theoretically rich, its practical performance declines when only a single tree is extracted from the mixture. Nonetheless, it remains a strong candidate for scalable applications.

These insights offer practical guidance for selecting learning algorithms based on data characteristics such as distributional assumptions, alphabet size, and dataset scale.

Acknowledgments. This work is supported by the Czech Science Foundation project 23-07299S (“Statistical Relational Learning in Dynamic Domains”).

Disclosure of Interests. The author has no competing interests to declare that are relevant to the content of this article.

References

1. Benavoli, A., Corani, G., Demšar, J., Zaffalon, M.: Time for a change: a tutorial for comparing multiple classifiers through Bayesian analysis. *J. Mach. Learn. Res.* **18**(77), 1–36 (2017). <http://jmlr.org/papers/v18/16-305.html>
2. Bhattacharyya, A., Gayen, S., John, P.G., Sen, S., Vinodchandran, N.V.: Distribution learning meets graph structure sampling (2024). <https://arxiv.org/abs/2405.07914>
3. Bhattacharyya, A., Gayen, S., Price, E., Vinodchandran, N.V.: Near-optimal learning of tree-structured distributions by chow-liu (2021). <https://arxiv.org/abs/2011.04144>
4. Cesa-Bianchi, N., Lugosi, G.: Prediction, Learning, and Games. Cambridge University Press (2006)
5. Chekuri, C., Vondrak, J., Zenklusen, R.: Dependent randomized rounding via exchange properties of combinatorial structures. In: 2010 IEEE 51st Annual Symposium on Foundations of Computer Science, pp. 575–584 (2010). <https://doi.org/10.1109/FOCS.2010.60>
6. Chickering, D.M., Meek, C., Heckerman, D.: Large-sample learning of Bayesian networks is NP-hard (2012). <https://arxiv.org/abs/1212.2468>
7. Chow, C., Liu, C.: Approximating discrete probability distributions with dependence trees. *IEEE Trans. Inf. Theory* **14**(3), 462–467 (1968). <https://doi.org/10.1109/TIT.1968.1054142>
8. Cooper, G.F.: The computational complexity of probabilistic inference using Bayesian belief networks. *Artif. Intell.* **42**(2), 393–405 (1990). [https://doi.org/10.1016/0004-3702\(90\)90060-D](https://doi.org/10.1016/0004-3702(90)90060-D), <https://www.sciencedirect.com/science/article/pii/S000437029090060D>

9. Heckerman, D., Nathwani, B.: Toward normative expert systems: Part II probability-based representations for efficient knowledge acquisition and inference. *Methods Inf. Med.* **31**, 106–116 (1992) <https://doi.org/10.1055/s-0038-1634868>
10. Imoto, S., Miyano, S., Matsuno, H.: Chapter 11 - gene networks: estimation, modeling, and simulation. In: Kriete, A., Eils, R. (eds.) *Comput. Syst. Biol.*, 205–228. Academic Press, Burlington (2006). <https://doi.org/10.1016/B978-012088786-6/50030-7>, <https://www.sciencedirect.com/science/article/pii/B9780120887866500307>
11. Koriche, F.: Online forest density estimation. In: *UAI* (2016)
12. Kotłowski, W., Grünwald, P.: Maximum likelihood vs. sequential normalized maximum likelihood in on-line density estimation. In: Kakade, S.M., von Luxburg, U. (eds.) *Proceedings of the 24th Annual Conference on Learning Theory. Proceedings of Machine Learning Research*, vol. 19, pp. 457–476. PMLR, Budapest, Hungary (09–11 Jun 2011). <https://proceedings.mlr.press/v19/kotlowski11a.html>
13. Lowd, D., Davis, J.: Learning markov network structure with decision trees. In: *Proceedings of the 2010 IEEE International Conference on Data Mining*. p. 334–343. ICDM '10, IEEE Computer Society, USA (2010). <https://doi.org/10.1109/ICDM.2010.128>, <https://doi.org/10.1109/ICDM.2010.128>
14. Meila, M., Jordan, M.I.: Learning with mixtures of trees. *J. Mach. Learn. Res.* **1**, 1–48 (2001). <https://doi.org/10.1162/153244301753344605>
15. Pearl, J.: *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (1988)
16. Selvam, N.R., Zhang, H., den Broeck, G.V.: Mixtures of all trees (2023). <https://arxiv.org/abs/2302.14202>
17. Shtar'kov, Y.M.: Universal sequential coding of single messages. *Problemy Peredachi Informatsii* **23**(3), 3–17 (1987)
18. Van Haaren, J., Davis, J.: Markov network structure learning: a randomized feature generation approach. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 2, pp. 1148–1154 (2012). <https://doi.org/10.1609/aaai.v26i1.8315>